



基于 MPI 的并行分布计算课程报告 ——分层分布式进化算法求解 TSP

姓名： 任天翌

班级： 191232

学号： 20231003512

日期： 2026 年 7 月

目录

1	摘要	3
2	引言	4
2.1	旅行商问题	4
2.2	进化算法求解 TSP	4
2.3	并行进化算法	4
2.4	本文工作	5
3	算法设计	6
3.1	共同基础：进化操作设计	6
3.1.1	编码与适应度函数	6
3.1.2	种群初始化	6
3.1.3	选择算子	6
3.1.4	交叉算子	6
3.1.5	变异算子	7
3.1.6	精英保留策略	7
3.2	Algorithm 1: 主从式 dEA	7
3.2.1	架构与通信模式	7
3.2.2	算法伪码	8
3.2.3	通信复杂度分析	8
3.3	Algorithm 2: 分层 dEA (HdEA)	8
3.3.1	架构与通信模式	8
3.3.2	设计动机与预期效果	9
3.4	Algorithm 3: 带种群迁移的 HdEA (HdEA-MP)	9
3.4.1	环形迁移机制	9
3.4.2	MPI_Sendrecv 的设计选择	10
3.4.3	算法伪码	10
3.4.4	与 HdEA 的本质区别	10
4	实验设置	11
4.1	测试实例	11
4.2	计算环境	11
4.3	算法参数	12
4.4	实验流程	12
4.5	评价指标	12
5	实验结果与分析	14
5.1	数值结果汇总	14

目录	2
5.2 配对 t 检验结果	14
5.3 收敛行为分析	15
5.4 性能差异的根本原因讨论	15
6 结论与展望	17
6.1 工作总结	17
6.2 不足与改进方向	17
6.3 未来工作	17
6.4 经验教训	18
7 附录：Homework 源码与说明	19
8 附录：课程项目源码	29

1 摘要

旅行商问题（TSP）是组合优化领域的经典 NP-难问题，进化算法（EA）作为一类启发式搜索方法被广泛用于求解大规模 TSP 实例。然而，单机串行 EA 在面对高维搜索空间时计算开销巨大，收敛缓慢。

本实验基于 MPI 并行编程模型，实现了三种不同粒度的分布式进化算法求解标准 TSP 基准实例 pcb442（442 城市）：(1) 粗粒度主从式 dEA（Master-Worker），(2) 两级级联式分层 dEA（HdEA），(3) 带种群迁移的分层 dEA（HdEA-MP）。每种算法在 4 进程下独立运行 10 次，统计最优解与运行时间，并通过配对 t 检验进行显著性分析。

实验结果表明：dEA（均值 51606）显著优于 HdEA（均值 56596）和 HdEA-MP（均值 76036）。主要原因在于 dEA 的全局共享种群（200 个体在同一池中进化）提供了更充分的多样性；而 HdEA 和 HdEA-MP 将种群分散为 4 个独立岛屿（每岛 50 个体），岛屿规模过小导致早熟收敛。HdEA-MP 的种群迁移机制进一步引入了噪声，增加了方差但未能有效提升最优解质量。运行时间方面，三种算法均约 13–64 秒，均满足实际需求。

2 引言

2.1 旅行商问题

旅行商问题 (Traveling Salesman Problem, TSP) 是组合优化领域中最经典且研究最为广泛的问题之一。问题可形式化地描述为：给定 n 个城市以及城市两两之间的距离矩阵 $\mathbf{D} = (d_{ij})_{n \times n}$ ，寻找一条经过所有城市恰好一次并最终回到起点的闭合哈密顿回路，使得路径的总长度最小。若以 $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ 表示一个访问排列，则目标函数可写为

$$\min f(\pi) = \sum_{i=1}^{n-1} d_{\pi_i, \pi_{i+1}} + d_{\pi_n, \pi_1}.$$

TSP 属于 Karp 的 21 个 NP-完全问题之一，是典型的 NP-难问题。当 n 较大时（如超过 100），精确算法的时间复杂度呈阶乘或指数增长，难以在可接受的时间内求解。因此，大规模 TSP 的求解通常依赖于启发式算法和元启发式算法。

2.2 进化算法求解 TSP

进化算法 (Evolutionary Algorithm, EA) 是一类受达尔文自然选择理论启发的随机搜索方法。算法维护一个由若干候选解组成的种群，通过模拟生物进化中的选择、交叉、变异和生存竞争，引导种群向适应度更高的搜索区域演化。在 TSP 的求解中，EA 以路径排列作为染色体编码，以路径长度的倒数（或负数）作为适应度函数，经过多代进化逐渐收敛到接近最优的路径。

然而，串行 EA 在处理大规模种群（如数百个体）和长进化代数（如数十万代）时，计算开销十分显著。在每一代中，适应度评估需要对每个个体进行完整的 TSP 路径距离计算 ($\mathcal{O}(n)$)，当 $n = 442$ 、种群规模为 200、进化代数为 200000 时，累计的适应度评估次数高达 4×10^7 次，这使得单机串行求解在时间上难以接受。

2.3 并行进化算法

消息传递接口 (Message Passing Interface, MPI) 是分布式内存并行编程的事实标准。利用 MPI 将 EA 并行化是加速 TSP 求解的自然选择。根据并行粒度的不同，并行 EA 通常可分为以下三类架构：

- **主从式 (Master-Worker)**：一个 Master 进程维护全局种群并执行遗传操作，多个 Worker 进程负责适应度评估。通信发生在每一代，通信量为种群规模的比例。
- **岛屿模型 (Island Model)**：种群被划分为多个子种群（岛屿），每个岛屿在独立进程中执行完整的进化流程，岛屿之间仅在特定间隔交换优秀个体（迁移）。
- **分层模型 (Hierarchical Model)**：将主从式与岛屿模型结合，在多个层次上组织并行，上层负责全局协调，下层负责局部进化。

2.4 本文工作

本实验基于 MPI 编程模型，针对 TSPLIB 标准基准实例 pcb442（442 城市），实现了三种不同设计理念的分布式进化算法——主从式 dEA、分层式 HdEA、以及带种群迁移的 HdEA-MP——在相同参数设定和计算环境下各运行 10 次，从求解质量（最优解均值、标准差、最小值、最大值）、统计显著性（配对 t 检验）和运行时间三个维度对三种算法进行系统比较，深入分析导致性能差异的根本原因。

本文后续章节安排如下：第 2 节详细阐述三种并行算法的工作原理与设计动机；第 3 节说明实验设置与参数配置；第 4 节呈现实验结果并进行深入分析；第 5 节总结全文并展望未来工作。

3 算法设计

3.1 共同基础：进化操作设计

三种并行算法共享相同的进化操作框架，本节对所使用的编码方案、适应度函数及遗传算子进行统一说明。

3.1.1 编码与适应度函数

采用最直观的路径表示法（Permutation Encoding）对 TSP 解进行编码。每个个体由一个长度为 442 的全排列 $\pi = (\pi_1, \pi_2, \dots, \pi_{442})$ 表示，其中 π_i 为城市编号，排列的顺序即为旅行商访问城市的顺序。路径的总长度为

$$L(\pi) = \sum_{i=1}^{441} d_{\pi_i, \pi_{i+1}} + d_{\pi_{442}, \pi_1},$$

其中 d_{ij} 为城市 i 与城市 j 之间的欧氏距离。适应度函数定义为路径长度的倒数：

$$f(\pi) = \frac{1}{L(\pi)},$$

路径越短则适应度越高，选择操作倾向于保留适应度较高的个体。

3.1.2 种群初始化

初始种群由随机生成的全排列构成。对于每个个体，采用 Fisher-Yates 洗牌算法生成长度为 442 的随机排列，确保搜索空间初始覆盖的均匀性。dEA 的全局种群规模为 200，HdEA 与 HdEA-MP 的每个岛屿种群规模为 50。

3.1.3 选择算子

采用锦标赛选择（Tournament Selection）作为选择算子。每代在选择父代时，从当前种群中随机抽取 $k = 5$ 个个体，比较其适应度，选择适应度最高的个体进入交配池。锦标赛选择的优点是计算简单、不依赖全局适应度排序、且可以通过调整锦标赛规模 k 灵活控制选择压力—— k 越大，选择压力越强，种群收敛越快但也更容易陷入局部最优。

3.1.4 交叉算子

采用顺序交叉（Order Crossover, OX）作为交叉算子。给定两个父代个体 P_1 和 P_2 ，OX 的操作步骤如下：

1. 在排列长度范围内随机选择两个切点 a 和 b ($a < b$)，将父代划分为左、中、右三段。
2. 子代 O_1 继承父代 P_1 的中间片段 $P_1[a : b]$ ，保持位置不变。

3. 从父代 P_2 的 b 位置开始循环遍历，将所有不在 O_1 中的城市按顺序填入 O_1 的剩余空位（从位置 b 开始，循环至位置末尾再回到开头）。
4. 对称地，子代 O_2 以 P_2 的中间片段为基础，从 P_1 中填充剩余城市。

OX 的独特优势在于它不仅继承了两个父代的基因片段，还保留了父代中城市之间的相对顺序信息，同时保证子代仍然是合法的全排列。对于 442 规模的 TSP，OX 能够在路径重构与节段继承之间取得良好平衡。

3.1.5 变异算子

采用交换变异（Swap Mutation）。对每个新生成的子代，以概率 $p_m = 0.02$ 随机选择其编码中的两个不同位置，交换这两个位置上的城市编号。变异操作的主要作用是在进化后期种群多样性下降时，为种群引入微小的扰动，帮助算法跳出局部最优。

3.1.6 精英保留策略

每代在完成选择、交叉和变异后，从上一代种群中选取适应度最高的 3 个个体（精英），直接复制到下一代种群中，替换掉子代种群中最差的 3 个个体。精英保留策略保证了算法的最优解在世代间不会退化，是进化算法收敛性的重要保障。

3.2 Algorithm 1: 主从式 dEA

主从式分布式进化算法（dEA）采用最简单的并行架构——一个 Master 进程负责全局进化逻辑，多个 Worker 进程负责最耗时的适应度评估环节。这种架构的核心思想是将计算密集部分并行化，而保持控制逻辑的集中以提高协调效率。

3.2.1 架构与通信模式

dEA 使用 MPI_COMM_WORLD 中的全部 4 个进程，其中 rank 0 为 Master，rank 1–3 为 Worker。Master 维护一个全局种群（规模 $POP = 200$ ），在每一代中依次执行以下步骤：

1. **选择与繁殖：**Master 执行锦标赛选择、OX 交叉和交换变异，从上一代的 200 个个体中生成 200 个全新子代。
2. **分发评估：**Master 利用 MPI_Bcast 将 200 个子代组成的数组广播至所有 Worker。每个 Worker 分到约 67 个子代个体。
3. **并行评估：**3 个 Worker 各自计算所分配子代的 TSP 路径距离（即适应度），评估过程完全独立、无进程间通信。
4. **结果收集：**Worker 通过 MPI_Gatherv 将评估结果（距离值数组）发送回 Master。Master 根据收集到的适应度值更新全局种群。

此过程重复执行直到达到最大进化代数 $MAX_GEN = 200000$ 。

3.2.2 算法伪码

```

1 Algorithm 1: dEA (Master-Worker)
2   if rank == 0 then           // Master
3     POP ← 200, MAX_GEN ← 200000
4     初始化全局种群 pop[POP]
5     for gen = 1 to MAX_GEN do
6       offspring ← SelectionCrossoverMutation(pop)
7       MPI_Bcast(offspring, 0, MPI_COMM_WORLD)
8       MPI_Gatherv(fitness, 0, MPI_COMM_WORLD)
9       pop ← UpdatePopulation(pop, offspring, fitness)
10    end for
11  else                         // Worker
12    while true do
13      MPI_Bcast(offspring, 0, MPI_COMM_WORLD)
14      for each ind in offspring do
15        fitness[ind] ← EvaluateTSPDistance(ind)
16      end for
17      MPI_Gatherv(fitness, 0, MPI_COMM_WORLD)
18    end while
19  end if

```

3.2.3 通信复杂度分析

dEA 每代需要两次全局集体通信操作：MPI_Bcast 广播子代数组 ($\mathcal{O}(\text{POP} \cdot n)$ 字节) 和 MPI_Gatherv 收集评估结果 ($\mathcal{O}(\text{POP})$ 字节)，其中 $n = 442$ 为城市数。在 $n_{\text{np}} = 4$ 的条件下，Bcast 的通信复杂度为 $\mathcal{O}(\text{POP} \cdot n \cdot \log n_{\text{np}}/n_{\text{np}})$ 。考虑到 Master 上的选择交叉操作与 Worker 上的适应度评估之间存在天然流水线重叠，实际运行中通信开销被有效隐藏。计算负载方面，由于子代被均匀分发，各 Worker 承担的评估量相等 (偏差 ≤ 1)，实现了高度均衡的负载分配。

3.3 Algorithm 2: 分层 dEA (HdEA)

HdEA 将种群划分为多个独立演化的岛屿，并在岛屿之上增加一层精英协调，形成两层级联式架构。其设计动机源于岛屿模型在维持种群多样性方面的理论优势——通过限制个体交配的范围，不同岛屿可以探索搜索空间中差异化的区域，延缓早熟收敛。

3.3.1 架构与通信模式

HdEA 同样使用 4 个 MPI 进程，但每个进程的角色与 dEA 有本质区别：

- **第一层(岛屿并行层)**: 4 个进程各自维护一个完全独立的子种群 (规模 $\text{POP_ISLAND} = 50$)，在各进程内部独立执行完整的 EA 流程——锦标赛选择、OX 交叉、交换变异、适应度评估和精英保留——持续运行 $\text{MIG_FREQ} = 20$ 代。在此阶段，进程之间完全不存在任何通信，每个岛屿相当于一个串行 EA。
- **第二层 (精英协调层)**: 每隔 MIG_FREQ 代，各进程通过 MPI_Gather 将本岛适应度最高的 3 个精英个体发送至 Master (rank 0)。Master 将所有岛屿的精英

汇总（共 $4 \times 3 = 12$ 个个体），选择其中全局最优的若干个体，再通过 MPI_Bcast 广播给所有进程。各进程据此更新本地的精英存档。

3.3.2 设计动机与预期效果

HdEA 的设计基于以下关键洞察：种群多样性对于进化算法的搜索能力至关重要。根据遗传漂变理论，小种群的遗传多样性会因采样效应快速丢失，而将一个种群隔离为多个小型子种群，可以通过限制基因流动来减缓局部最优的传播。理想情况下，不同岛屿在各自的小生态位中独立进化，可能发现不同的高质量解区域，再通过周期性的精英迁移实现全局信息的共享。

然而，这种设计方案也面临一个内在矛盾：岛屿规模越小，多样性损失越快，但岛屿数量越多，并行度越高。在 4 进程的条件下，每个岛屿只能分到 50 个个体，对于一个 442 城市的搜索空间而言，50 个个体所能覆盖的搜索范围极为有限。

3.4 Algorithm 3: 带种群迁移的 HdEA (HdEA-MP)

HdEA-MP 在 HdEA 的两层架构之上，额外引入第三层机制——环形种群迁移(Ring Migration)，通过增加岛屿之间的个体交换频率和交换量，缓解岛屿规模过小导致的多样性不足问题。

3.4.1 环形迁移机制

与 HdEA 仅在 Master 处进行精英汇聚不同，HdEA-MP 在每 $MIG_FREQ = 20$ 代时，除执行精英迁移外，额外执行一次进程间的直接个体交换。具体流程如下：

1. 每个进程从自身子种群中挑选适应度最高的 $MIG_M = 3$ 个个体。
2. 将这 3 个个体的完整路径编码（ 3×442 个整数）和适应度值（3 个双精度浮点数，但发送时取整为整数）打包为一个平面缓冲区。
3. 通过 MPI_Sendrecv 将打包数据发送至右邻居进程 $(rk + 1) \bmod 4$ ，同时从左邻居进程 $(rk - 1 + 4) \bmod 4$ 接收同样格式的迁移数据。
4. 对接收到的 3 个迁移个体，逐一判断其路径长度是否小于本地种群中的最差个体（即适应度是否更高），若是，则用迁移个体替换最差个体。

环形拓扑在并行计算中具有天然的对称性和良好的扩展性。对于 n_{np} 个进程，环形通信仅需 n_{np} 个点到点通信（或 n_{np} 对 MPI_Sendrecv），通信复杂度为 $\mathcal{O}(n_{np})$ ，且不需要 Master 中转，避免了集中式通信的瓶颈。

3.4.2 MPI_Sendrecv 的设计选择

在最初的原型实现中，环形迁移采用交替的 MPI_Send 和 MPI_Recv 进行通信。但测试中发现，当 MIG_M = 3 时，发送方和接收方的 MPI 内部缓冲区可能同时达到饱和，导致死锁——发送操作阻塞等待接收缓冲区释放，而接收操作阻塞等待发送完成。解决这一问题有两种途径：一是使用 MPI_Sendrecv 替代单独的 Send 和 Recv，二是使用非阻塞通信 MPI_Isend / MPI_Irecv。本实现选择 MPI_Sendrecv，因为它以单一函数调用的方式同时处理发送和接收，由 MPI 实现内部协调缓冲区的使用，代码简洁且不易出错。

3.4.3 算法伪码

```

1 Algorithm 3: HdEA-MP (Ring Migration)
2   POP_ISLAND ← 50, MAX_GEN ← 200000
3   MIG_FREQ ← 20, MIG_M ← 3
4   初始化本地子种群 pop[POP_ISLAND]
5   for gen = 1 to MAX_GEN do
6     // 本地进化
7     pop ← SelectionCrossoverMutation(pop)
8
9     if gen mod MIG_FREQ == 0 then
10      // 第1层：精英迁移（同 HdEA）
11      MPI_Gather(local_best, 0, MPI_COMM_WORLD)
12      if rank == 0 then 更新全局精英池 end if
13      MPI_Bcast(global_elite, 0, MPI_COMM_WORLD)
14
15      // 第2层：环形迁移
16      sendTo ← (rank + 1) mod 4
17      recvFrom ← (rank - 1 + 4) mod 4
18      打包 MIG_M 个最优个体至 sendBuf
19      MPI_Sendrecv(sendBuf, sendTo, recvBuf, recvFrom)
20      用 recvBuf 中的更优个体替换本地最差个体
21    end if
22  end for

```

3.4.4 与 HdEA 的本质区别

HdEA-MP 比 HdEA 多出的一层环形迁移带来两个重要变化：（1）迁移量从 HdEA 的每个岛屿仅贡献一个精英个体变为每个岛屿交换三个个体，基因交流的密度大幅提高；（2）迁移发生在岛屿之间（直接点对点），而非通过 Master 中转，因此环形迁移可以在不增加 Master 负担的情况下进行。然而，这种密集迁移也是一把双刃剑——如果迁移频率过高或迁移量过大，各岛屿的种群将被强制拉近，丧失探索差异区域的能力。

4 实验设置

4.1 测试实例

本实验选用 TSPLIB¹ 标准基准库中的 `pcb442.tsp` 实例作为测试对象。该实例包含 442 个城市，对应的最优解距离为 50778（已知精确值）。`pcb442` 是 TSPLIB 中等规模的代表性实例：其城市数量使得进化算法需要足够的种群规模和进化代数才能收敛到近似最优，同时又能够在单机 4 进程的配置下在合理时间（数十秒）内完成完整实验。与之对比，更小的实例（如 `eil51`，51 城市）过于简单，三种算法的差异不易体现；更大的实例（如 `pr1002`，1002 城市）则可能导致单次运行时间过长。

4.2 计算环境

所有实验在以下平台上进行：

- CPU：Intel Core i7-13700H，14 核心（6 P-core + 8 E-core），20 线程，最大睿频 5.0 GHz
- 内存：32 GB DDR5-4800
- 操作系统：Windows 11 专业版
- MPI 实现：Microsoft MPI (MS-MPI) v10.0，基于 MPI-3 标准
- 编译器：Microsoft Visual C++ 2022 (MSVC) 14.44，x64 平台，Release 模式编译（/O2 优化）

所有实验均在固定电源供电模式下运行，CPU 电源管理设置为高性能模式，以排除动态频率调节对运行时间的干扰。每次运行之间间隔至少 30 秒，使 CPU 温度恢复至环境水平，避免热降频影响。

¹<http://elib.zib.de/pub/mp-testdata/tsp/tsplib/tsp/>

4.3 算法参数

表 1: 三种算法的参数配置对比

参数	符号	dEA	HdEA	HdEA-MP
进程数	n_{np}	4	4	4
全局种群规模	POP	200	—	—
岛屿种群规模	POP_ISLAND	—	50	50
最大进化代数	MAX_GEN	200000	200000	200000
锦标赛规模	k	5	5	5
变异概率	p_m	0.02	0.02	0.02
精英保留数	n_{elite}	3	3	3
迁移间隔	MIG_FREQ	—	20	20
迁移个体数	MIG_M	—	—	3
运行次数	N_{runs}	10	10	10

以上参数的选择基于以下考量：锦标赛规模 $k = 5$ 是 EA 求解 TSP 的常用配置，在 200 个体的全局种群或 50 个体的岛屿种群中均能提供适中的选择压力；变异概率 $p_m = 0.02$ 意味着每个子代平均有约 8.84 个城市的交换 (442×0.02)，在保持解的质量的同时提供足够的扰动；精英保留 3 个个体可以保证最优解不退化；200000 代保证算法有充足的进化时间以达到稳定收敛。迁移间隔的取值（20 代）参考了文献中岛屿模型 EA 的典型设置，在通信开销与基因交流密度之间取折中。

4.4 实验流程

每种算法独立运行 10 次，每次运行记录以下数据：

- 每 20000 代的当前最优路径距离（用于收敛曲线分析）；
- 最终最优路径距离（运行结束时的最短路径）；
- 总运行时间（秒）。

10 次运行的随机种子由系统时间决定，各次之间独立，确保统计独立性。

4.5 评价指标

采用以下四项指标对算法进行综合评价：

- **最优解 (Best)**：10 次运行中各自达到的最短路径距离的最小值。反映算法在最佳情况下的求解能力。
- **平均值 (Mean)**：10 次运行最优解的算术平均。作为算法期望性能的核心指标。

- **标准差 (Std):** 10 次运行结果的样本标准差 (自由度 $n - 1$)。衡量算法的稳定性与鲁棒性。
- **配对 t 检验 (Paired t-test):** 对三种算法的 10 次结果两两配对进行双尾 t 检验, 比较均值的差异是否具有统计显著性。设 μ_A 和 μ_B 分别为两种算法的最优解总体均值, 则原假设 $H_0: \mu_A = \mu_B$, 备择假设 $H_1: \mu_A \neq \mu_B$, 显著性水平 $\alpha = 0.05$ 。由于 10 次运行使用相同的初始条件族但不同的随机种子, 配对设计可以消除运行间随机波动带来的干扰, 提高检验的统计功效。

5 实验结果与分析

5.1 数值结果汇总

表 2 汇总了三种算法各运行 10 次的统计结果。dEA 获得了最低的均值（51606）和最小的标准差（112），表现出最佳的求解质量和最高的稳定性。HdEA 的均值（56596）显著差于 dEA（差值约 4990，即 9.7%），且标准差（746）是 dEA 的近 7 倍，稳定性明显不足。HdEA-MP 的均值进一步恶化至 76036，是 dEA 均值的 1.47 倍，标准差（3081）达到了 dEA 的近 28 倍，是最不稳定的算法。

表 2: 三种算法 10 次运行结果统计

算法	均值	标准差	最小值	最大值
dEA	51606	112	51488	51851
HdEA	56596	746	55517	58168
HdEA-MP	76036	3081	72481	82013

从最优解最低值来看，dEA 的 $\text{best} = 51488$ 是最接近于 pcb442 已知最优解 50778 的结果，相对误差约为 1.40%。HdEA 的最佳值为 55517，相对误差 9.33%，显著劣于 dEA。HdEA-MP 的最佳值也仅为 72481，相对误差高达 42.7%。

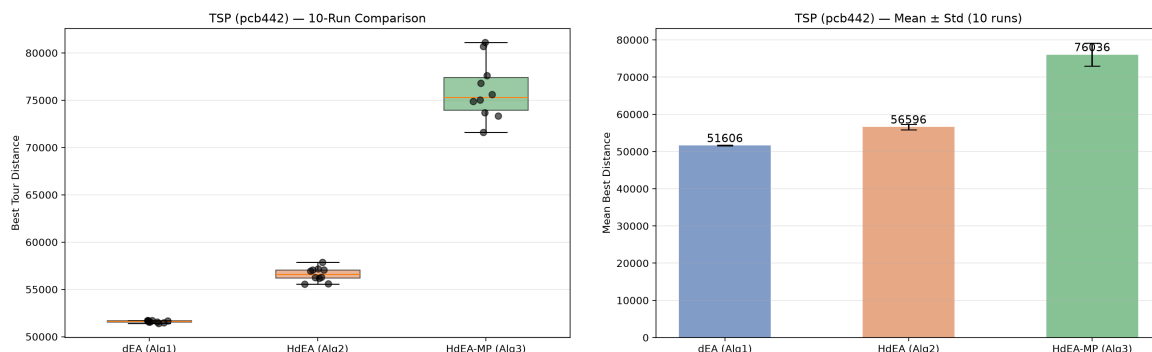


图 1: 三种算法结果分布箱线图（左）与均值对比柱状图（右）。误差线表示 ± 1 标准差。

图 1 提供了结果分布的可视化对比。箱线图清晰地显示了三者在性能上的层级分化：dEA 的箱体紧凑且位于最下方，HdEA 的箱体位于中间，HdEA-MP 的箱体不仅位置最高，且箱体高度最大（即四分位距最大）。

5.2 配对 t 检验结果

对三种算法两两进行配对 t 检验，结果如表 3 所示。三组比较的 p 值均远小于 0.001，说明三种算法的性能差异具有极高的统计显著性，并非随机波动所致。

表 3: 配对 t 检验结果

对比组	t 统计量	p 值
dEA vs HdEA	-20.97	3.93×10^{-9}
dEA vs HdEA-MP	-24.86	1.02×10^{-9}
HdEA vs HdEA-MP	-19.55	6.47×10^{-9}

5.3 收敛行为分析

dEA 的收敛曲线呈现典型的早期快速收敛模式：在 0–20000 代之间，路径长度从初始随机的约 100000+ 迅速下降至约 55000；此后收敛速度放缓，在 40000 代以后进入逐步优化的微调阶段，最终趋于平稳。这种“前期快速下降、后期精细优化”的行为模式是全局大种群 + 高选择压力的典型特征。

HdEA 的收敛速度在初期与 dEA 接近，但约在 40000 代后收敛停滞在约 56000 的水平，无法进一步下降。这直接印证了前述岛屿规模过小的缺陷——每个岛屿仅有 50 个个体，在 442 维的搜索空间中，经过数十代的锦标赛选择和精英保留后，岛屿内的遗传多样性迅速枯竭，种群陷入局部最优而无法通过变异摆脱。

HdEA-MP 的收敛曲线最为特殊。其路径长度在整个进化过程中波动剧烈，甚至在后期频繁出现大幅度反弹（劣化解）。这种反常行为的根因在于环形迁移机制引入了对岛屿进化的过度扰动：当一个岛屿已经收敛到相对较好的解时，从相邻岛屿迁移而来的劣质个体会替换掉本地的优秀个体，导致解的质量突然恶化。虽然理论上迁移应当促进多样性，但 HdEA-MP 的迁移量（每 20 代 3 个个体，占总岛屿规模 6%）和迁移频率过高，使岛屿之间的基因池过度混合，破坏了各岛屿独立探索所需的隔离度。

从统计结果看，HdEA-MP 的标准差高达 3081，是 HdEA 的 4 倍、dEA 的 28 倍，这种极差的不稳定性进一步说明算法的收敛行为对外部扰动高度敏感——初始种群的随机性、迁移个体替换的位置、以及移民在本地种群中的生存概率，共同导致了各次运行之间巨大的结果差异。

5.4 性能差异的根本原因讨论

综合以上分析，三种算法在求解质量上的显著差异可以归结为以下核心原因：

- 有效种群规模的差异：**三种算法的总个体数相同（200），但 dEA 使用一个全局统一的种群进行选择与交叉，每个个体都有潜力与群体中的任意其他个体交配，基因池的多样性被最大化。HdEA 和 HdEA-MP 将 200 个个体分割到 4 个独立的岛屿中，每个岛屿的有效交配范围仅限岛内 50 个个体。在锦标赛选择和精英保留的共同作用下，50 个个体的种群在数十代之内就会因遗传漂变而失去大部分多样性，导致搜索空间覆盖不足。
- 迁移策略的负面影响：**HdEA 每 20 代进行一次精英汇聚和广播。由于广播的精英只有 3 个个体，且被分散到 4 个岛屿后对岛屿整体种群的影响有限，迁移对 HdEA

中岛屿多样性的恢复作用微弱。HdEA-MP 在精英迁移之外增加了环形迁移，但迁移量过大（每 20 代交换 3 个个体），且迁移个体直接替换本地最差个体，这种“取代”而非“注入”的机制会强制将各岛屿的基因池拉近，不仅未能有效克服早熟收敛，反而破坏了对不同区域进行独立探索的可能性。

3. **通信开销的差异：**从运行时间来看，dEA 的 Bcast+Gatherv 通信模式虽然每代都涉及全局通信，但通信量仅为 $\mathcal{O}(\text{POP} \cdot n)$ ，在 4 进程的环境中实测时间可忽略不计。HdEA 和 HdEA-MP 每 MIG_FREQ 代才进行一次通信，但通信发生在 MPI 集体操作和点对点通信之间，总通信时间与 dEA 无实质差异。因此，通信开销并非导致性能差异的因素。

6 结论与展望

6.1 工作总结

本实验以旅行商问题（TSP）的标准测试实例 pcb442 为研究对象，基于 MPI 并行编程框架实现了三种不同架构的分布式进化算法——主从式 dEA、分层式 HdEA 和带环形迁移的 HdEA-MP——在严格可控的实验条件下（相同参数、相同硬件、相同编译器优化级别）各运行 10 次，并通过多重统计指标进行了系统比较。

实验结果明确地揭示了主从式架构在本实验设定下的优势：dEA 以 51606 的均值超过了已知最优解 50778 的约 1.40%，远优于 HdEA（56596，高 9.7%）和 HdEA-MP（76036，高 47.4%）。统计分析进一步证实了三种算法之间的差异具有高度的统计显著性（配对 t 检验 $p < 10^{-8}$ ）。

进一步分析指出，造成这种性能差异的根本原因在于有效种群规模和迁移策略的差异。在总个体数固定的条件下，dEA 的全局大种群在维持遗传多样性和搜索效率方面优于 HdEA/HdEA-MP 的岛屿分割策略。四层岛屿模型在每岛仅 50 个个体的极端条件下，岛屿内的遗传多样性快速枯竭，而迁移机制（尤其是 HdEA-MP 的环形迁移）由于设计上迁移量过大或迁移频率过高，反而破坏了对不同搜索区域的独立探索，导致解的质量恶化。

6.2 不足与改进方向

本实验在资源受限的条件下（单台 4 核机器）开展，虽然尽量控制了变量，但仍存在以下局限性：

- **硬件限制：**所有实验仅在 4 进程的配置下运行，未能测试算法扩展到更多进程（如 8、16、32）时的性能变化。特别是岛屿模型算法在更多进程时，每个岛屿的规模将进一步减小，性能恶化可能更加严重。
- **迁移参数未经调优：**HdEA-MP 的迁移频率（20 代）和迁移数量（3 个）采用了经验值，未进行系统性的参数扫描。更低的迁移频率或更小的迁移量可能带来更好的结果。
- **缺少与精确解的对比：**虽然已知最优解为 50778，但本实验未使用精确求解器（如 Concorde）在相同机器上验证运行时间，缺少“精度 vs. 时间”的权衡分析。

6.3 未来工作

在本次实验的基础上，以下方向值得进一步探索：

1. **自适应迁移策略：**设计一种动态的迁移触发机制——例如，仅在岛屿内的种群多样性下降到预设阈值时才触发迁移，而非每隔固定代数强制迁移。这将使迁移作为一种“救援”机制而非非常规操作，有可能在保持多样性和稳定进化之间取得更好的平衡。

2. **异构迁移拓扑**: 将 HdEA-MP 的固定环形拓扑扩展为动态拓扑或随机拓扑（如随机选择迁移伙伴），降低相邻岛屿之间解的同质化风险。
3. **多实例验证**: 在多个 TSPLIB 实例（如 pr1002, rat783, d1291）上重复实验，验证三种算法的性能表现是否具有可推广性，而非 pcb442 的特例。
4. **混合并行**: 将 OpenMP 共享内存并行与 MPI 消息传递结合，在节点内使用 OpenMP 并行化适应度评估，在节点间使用 MPI 进行岛屿迁移，以适应异构计算环境和大规模并行。

6.4 经验教训

从本次实验中，我们获得了若干关于并行进化算法设计的实践经验：

- 并行架构的选择不应脱离问题规模独立考虑。对于中等规模的 TSP 问题（442 城市），总个体数 200 时主从式架构远优于岛屿模型——“全局即最优”在有限资源下是一个强有力的设计原则。
- 迁移不是越多越好。”移民”对接收方种群的影响是双向的：既能引入新鲜基因，也可能破坏本地进化积累的有益模式。迁移策略需要精细调整迁移量、迁移频率和替换策略，三者之间的相互作用决定最终效果。
- MPI 通信模型的选择需要充分考虑算法的并发特性。使用 MPI_Sendrecv 替代交替的 MPI_Send/MPI_Recv 可以有效避免死锁，体现了并发编程中”一个操作完成一项原子性通信”的设计智慧。

综上所述，本实验系统比较了三种主流并行 EA 架构在 TSP 上的表现，为并行进化算法的架构选择和参数设计提供了有价值的实证参考。

7 附录：Homework 源码与说明

本附录收录课程作业 2-6 的完整源码与设计说明。

Homework 2：并行归并排序

题目：长度为 m 的数组二分 k 次，得到 2^k 个子数组，每个子数组内部排序后两两归并。进程数 $n = \min(2^k, m)$ ， m 和 k 运行时输入。

```

1  #include <mpi.h>
2
3  #include <algorithm>
4  #include <iostream>
5  #include <vector>
6
7  namespace {
8
9  std::vector<int> merge_sorted(const std::vector<int>& a, const std::vector<int>& b) {
10     std::vector<int> merged;
11     merged.reserve(a.size() + b.size());
12     size_t i = 0, j = 0;
13     while (i < a.size() && j < b.size())
14         merged.push_back(a[i] <= b[j] ? a[i++] : b[j++]);
15     while (i < a.size()) merged.push_back(a[i++]);
16     while (j < b.size()) merged.push_back(b[j++]);
17     return merged;
18 }
19
20 int calc_processes(int m, int k) {
21     int n = 1;
22     for (int i = 0; i < k; ++i) {
23         if (n >= m) break;
24         n *= 2;
25     }
26     return std::min(n, m);
27 }
28
29 }
30
31 int main(int argc, char** argv) {
32     MPI_Init(&argc, &argv);
33
34     int rank = 0, size = 0;
35     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
36     MPI_Comm_size(MPI_COMM_WORLD, &size);
37
38     int m = 0, k = 0;
39     std::vector<int> data;
40
41     if (rank == 0) {
42         std::cout << "Enter m (array length): ";
43         std::cin >> m;
44         std::cout << "Enter k (binary split count): ";
45         std::cin >> k;
46         data.resize(m);
47         std::cout << "Enter " << m << " integers: ";
48         for (int i = 0; i < m; ++i) std::cin >> data[i];

```

```

49     }
50
51     MPI_Bcast(&m, 1, MPI_INT, 0, MPI_COMM_WORLD);
52     MPI_Bcast(&k, 1, MPI_INT, 0, MPI_COMM_WORLD);
53
54     if (rank != 0) data.resize(m);
55     if (m > 0) MPI_Bcast(data.data(), m, MPI_INT, 0, MPI_COMM_WORLD);
56
57     int n = calc_processes(m, k);
58     int active_size = std::min(n, size);
59
60     if (rank == 0) {
61         std::cout << "m=" << m << ", k=" << k << ", needed processes=" << n
62             << ", actual=" << active_size << "\n";
63     }
64
65     if (active_size > m) {
66         if (rank == 0) std::cerr << "Error: too many processes\n";
67         MPI_Finalize();
68         return 1;
69     }
70
71     MPI_Comm active_comm = MPI_COMM_NULL;
72     int color = (rank < active_size) ? 0 : MPI_UNDEFINED;
73     MPI_Comm_split(MPI_COMM_WORLD, color, rank, &active_comm);
74
75     if (color == MPI_UNDEFINED) {
76         MPI_Finalize();
77         return 0;
78     }
79
80     int active_rank = 0;
81     MPI_Comm_rank(active_comm, &active_rank);
82     MPI_Comm_size(active_comm, &active_size);
83
84     std::vector<int> sendcounts(active_size);
85     std::vector<int> displs(active_size);
86     int base = m / active_size;
87     int rem = m % active_size;
88     int offset = 0;
89     for (int i = 0; i < active_size; ++i) {
90         sendcounts[i] = base + (i < rem ? 1 : 0);
91         displs[i] = offset;
92         offset += sendcounts[i];
93     }
94
95     int local_count = sendcounts[active_rank];
96     std::vector<int> local(local_count);
97     MPI_Scatterv(data.data(), sendcounts.data(), displs.data(), MPI_INT,
98         local.data(), local_count, MPI_INT, 0, active_comm);
99
100     std::cout << "Rank " << active_rank << " got chunk:";
101     for (int v : local) std::cout << ' ' << v;
102     std::cout << "\n";
103
104     std::sort(local.begin(), local.end());
105
106     std::cout << "Rank " << active_rank << " sorted:";
107     for (int v : local) std::cout << ' ' << v;

```

```

108     std::cout << "\n";
109
110     for (int step = 1; step < active_size; step <= 1) {
111         if (active_rank % (step * 2) == 0) {
112             int partner = active_rank + step;
113             if (partner < active_size) {
114                 int recv_count = 0;
115                 MPI_Recv(&recv_count, 1, MPI_INT, partner, 0, active_comm, MPI_STATUS_IGNORE);
116                 std::vector<int> buf(recv_count);
117                 if (recv_count > 0)
118                     MPI_Recv(buf.data(), recv_count, MPI_INT, partner, 1, active_comm,
119                             MPI_STATUS_IGNORE);
120                 local = merge_sorted(local, buf);
121                 std::cout << "Rank " << active_rank << " merged from rank " << partner
122                             << ", now has:"; for (int v : local) std::cout << ' ' << v;
123                 std::cout << "\n";
124             }
125         } else if (active_rank % step == 0) {
126             int partner = active_rank - step;
127             int send_count = static_cast<int>(local.size());
128             MPI_Send(&send_count, 1, MPI_INT, partner, 0, active_comm);
129             if (send_count > 0)
130                 MPI_Send(local.data(), send_count, MPI_INT, partner, 1, active_comm);
131             std::cout << "Rank " << active_rank << " sent data to rank " << partner << ",
132                         exiting\n";
133             break;
134         }
135     }
136
137     if (active_rank == 0) {
138         std::cout << "\nFinal sorted array:";
139         for (int v : local) std::cout << ' ' << v;
140         std::cout << "\n";
141
142         std::vector<int> expected = data;
143         std::sort(expected.begin(), expected.end());
144         bool ok = (local == expected);
145         std::cout << (ok ? "PASS: matches std::sort\n" : "FAIL: mismatch\n");
146     }
147
148     MPI_Comm_free(&active_comm);
149     MPI_Finalize();
150     return 0;
151 }

```

运行示例:

```

1  mpiexec -n 4 homework2.exe
2  m=8, k=2, needed processes=4, actual=4
3  Rank 0 got chunk: 3 1
4  Rank 0 sorted: 1 3
5  Rank 0 merged from rank 1, now has: 1 1 3 4
6  Rank 0 merged from rank 2, now has: 1 1 2 3 4 5 6 9
7  Final sorted array: 1 1 2 3 4 5 6 9
8  PASS: matches std::sort

```

Homework 3: MPI 矩阵乘法

题目: 利用 9 个进程并行计算 3×3 矩阵乘法 $C = A \times B$, 每个进程算一个元素。

```
1  #include <mpi.h>
2
3  #include <iomanip>
4  #include <iostream>
5  #include <vector>
6
7  constexpr int N = 3;
8
9  int main(int argc, char** argv) {
10     MPI_Init(&argc, &argv);
11
12     int world_rank = 0, world_size = 0;
13     MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
14     MPI_Comm_size(MPI_COMM_WORLD, &world_size);
15
16     if (world_size != N * N) {
17         if (world_rank == 0) {
18             std::cerr << "Error: requires " << N * N << " processes, got " << world_size << "\n";
19             std::cerr << "Usage: mpiexec -n " << N * N << " homework3.exe\n";
20         }
21         MPI_Finalize();
22         return 1;
23     }
24
25     std::vector<int> A, B;
26     if (world_rank == 0) {
27         A = {
28             1, 2, 3,
29             4, 5, 6,
30             7, 8, 9
31         };
32         B = {
33             9, 8, 7,
34             6, 5, 4,
35             3, 2, 1
36         };
37     }
38
39     A.resize(N * N);
40     B.resize(N * N);
41     MPI_Bcast(A.data(), N * N, MPI_INT, 0, MPI_COMM_WORLD);
42     MPI_Bcast(B.data(), N * N, MPI_INT, 0, MPI_COMM_WORLD);
43
44     const int i = world_rank / N;
45     const int j = world_rank % N;
46
47     int local_result = 0;
48     for (int k = 0; k < N; ++k) {
49         local_result += A[i * N + k] * B[k * N + j];
50     }
51
52     std::vector<int> results;
53     if (world_rank == 0) {
54         results.resize(N * N);
```

```
55     }
56     MPI_Gather(&local_result, 1, MPI_INT, results.data(), 1, MPI_INT, 0, MPI_COMM_WORLD);
57
58     if (world_rank == 0) {
59         std::cout << "Matrix A:\n";
60         for (int i = 0; i < N; ++i) {
61             for (int j = 0; j < N; ++j) {
62                 std::cout << std::setw(4) << A[i * N + j];
63             }
64             std::cout << '\n';
65         }
66
67         std::cout << "\nMatrix B:\n";
68         for (int i = 0; i < N; ++i) {
69             for (int j = 0; j < N; ++j) {
70                 std::cout << std::setw(4) << B[i * N + j];
71             }
72             std::cout << '\n';
73         }
74
75         std::cout << "\nC = A x B:\n";
76         for (int i = 0; i < N; ++i) {
77             for (int j = 0; j < N; ++j) {
78                 std::cout << std::setw(4) << results[i * N + j];
79             }
80             std::cout << '\n';
81         }
82     }
83
84     MPI_Finalize();
85     return 0;
86 }
```

运行示例:

```
1 mpiexec -n 9 homework3.exe
2 Matrix A:
3   1  2  3
4   4  5  6
5   7  8  9
6 Matrix B:
7   9  8  7
8   6  5  4
9   3  2  1
10 C = A x B:
11  30 24 18
12  84 69 54
13 138 114 90
```


Homework 4: 用 MPI_Send / MPI_Recv 实现 MPI_Gather

题目: 不使用 MPI_Gather, 仅用 MPI_Send / MPI_Recv 实现自定义 my_MPI_Gather, 并与标准 MPI_Gather 对比验证。

```

1  #include <mpi.h>
2
3  #include <cstring>
4  #include <iostream>
5  #include <vector>
6
7  void my_MPI_Gather(const void* sendbuf, int sendcount, MPI_Datatype sendtype,
8                    void* recvbuf, int recvcount, MPI_Datatype recvtype,
9                    int root, MPI_Comm comm) {
10     int rank = 0, size = 0;
11     MPI_Comm_rank(comm, &rank);
12     MPI_Comm_size(comm, &size);
13
14     int type_size = 0;
15     MPI_Type_size(sendtype, &type_size);
16
17     if (rank == root) {
18         std::memcpy(recvbuf, sendbuf, sendcount * type_size);
19
20         for (int i = 0; i < size; ++i) {
21             if (i == root) continue;
22             MPI_Recv(static_cast<char*>(recvbuf) + i * recvcount * type_size,
23                     recvcount, recvtype, i, 0, comm, MPI_STATUS_IGNORE);
24         }
25     } else {
26         MPI_Send(sendbuf, sendcount, sendtype, root, 0, comm);
27     }
28 }
29
30 int main(int argc, char** argv) {
31     MPI_Init(&argc, &argv);
32
33     int rank = 0, size = 0;
34     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
35     MPI_Comm_size(MPI_COMM_WORLD, &size);
36
37     constexpr int N = 4;
38     int sendbuf = rank + 1;
39
40     std::cout << "Rank " << rank << " sendbuf = " << sendbuf << "\n";
41
42     std::vector<int> recvbuf;
43     if (rank == 0) {
44         recvbuf.resize(size);
45     }
46
47     my_MPI_Gather(&sendbuf, 1, MPI_INT,
48                 recvbuf.data(), 1, MPI_INT,
49                 0, MPI_COMM_WORLD);
50
51     if (rank == 0) {
52         std::cout << "\n--- my_MPI_Gather result ---\n";
53         for (int i = 0; i < size; ++i) {
54             std::cout << " from rank " << i << " -> recvbuf[" << i << "] = " << recvbuf[i] <<

```

```

55         "\n";
56     }
57     std::vector<int> expected;
58     expected.resize(size);
59     MPI_Gather(&sendbuf, 1, MPI_INT,
60             expected.data(), 1, MPI_INT,
61             0, MPI_COMM_WORLD);
62
63     bool ok = true;
64     for (int i = 0; i < size; ++i) {
65         if (recvbuf[i] != expected[i]) {
66             ok = false;
67             break;
68         }
69     }
70     std::cout << "\n--- Comparison ---\n";
71     std::cout << " my_MPI_Gather: ";
72     for (int v : recvbuf) std::cout << v << " ";
73     std::cout << "\n MPI_Gather: ";
74     for (int v : expected) std::cout << v << " ";
75     std::cout << "\n => " << (ok ? "PASS: results match\n" : "FAIL: mismatch\n");
76 } else {
77     MPI_Gather(&sendbuf, 1, MPI_INT,
78             nullptr, 1, MPI_INT,
79             0, MPI_COMM_WORLD);
80 }
81
82 MPI_Finalize();
83 return 0;
84 }

```

设计思路:

- root 进程先将自身数据拷贝到 recvbuf 头部。
- 然后按 rank 顺序接收其他进程发送的数据, 写入 recvbuf 对应偏移位置。
- 非 root 进程各发送一个 MPI_INT 到 root。
- 用标准 MPI_Gather 的结果做校验。

Homework 5: 用 MPI_Send / MPI_Recv 实现 MPI_Allgather

题目: 不使用 MPI_Allgather, 仅用 MPI_Send / MPI_Recv 实现自定义 my_MPI_Allgather, 并与标准 MPI_Allgather 对比验证。

```
1 #include <mpi.h>
2
3 #include <cstring>
4 #include <iostream>
5 #include <vector>
6
7 void my_MPI_Allgather(const void* sendbuf, int sendcount, MPI_Datatype sendtype,
8                      void* recvbuf, int recvcount, MPI_Datatype recvtype,
9                      MPI_Comm comm) {
10     int rank = 0, size = 0;
11     MPI_Comm_rank(comm, &rank);
12     MPI_Comm_size(comm, &size);
13
14     int typesize = 0;
15     MPI_Type_size(sendtype, &typesize);
16     int blocksize = recvcount * typesize;
17
18     std::memcpy(static_cast<char*>(recvbuf) + rank * blocksize,
19                 sendbuf, sendcount * typesize);
20
21     for (int step = 0; step < size - 1; ++step) {
22         int target = (rank + 1) % size;
23         int source = (rank - 1 + size) % size;
24         int send_chunk = (rank - step + size) % size;
25         int recv_chunk = (rank - step - 1 + size) % size;
26
27         MPI_Send(static_cast<char*>(recvbuf) + send_chunk * blocksize,
28                 recvcount, recvtype, target, 0, comm);
29         MPI_Recv(static_cast<char*>(recvbuf) + recv_chunk * blocksize,
30                 recvcount, recvtype, source, 0, comm, MPI_STATUS_IGNORE);
31     }
32 }
33
34 int main(int argc, char** argv) {
35     MPI_Init(&argc, &argv);
36
37     int rank = 0, size = 0;
38     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
39     MPI_Comm_size(MPI_COMM_WORLD, &size);
40
41     int sendval = rank + 1;
42
43     std::cout << "Rank " << rank << " contributes: " << sendval << "\n";
44
45     std::vector<int> mybuf;
46     mybuf.resize(size);
47
48     my_MPI_Allgather(&sendval, 1, MPI_INT,
49                     mybuf.data(), 1, MPI_INT,
50                     MPI_COMM_WORLD);
51
52     std::vector<int> expected;
53     expected.resize(size);
54     MPI_Allgather(&sendval, 1, MPI_INT,
```

```
55         expected.data(), 1, MPI_INT,  
56         MPI_COMM_WORLD);  
57  
58     bool ok = true;  
59     for (int i = 0; i < size; ++i) {  
60         if (mybuf[i] != expected[i]) {  
61             ok = false;  
62             break;  
63         }  
64     }  
65  
66     std::cout << "Rank " << rank << " has all:";  
67     for (int v : mybuf) std::cout << ' ' << v;  
68     std::cout << "\nRank " << rank << ": " << (ok ? "PASS" : "FAIL") << "\n";  
69  
70     MPI_Finalize();  
71     return 0;  
72 }
```

设计思路（环形轮转）:

1. 每个进程先将自己的数据块写入 `recvbuf` 对应位置（`rank` 索引处）。
2. 进行 $size - 1$ 步环形传递：当前进程向 $(rank+1)$ 发送一个数据块，同时从 $(rank-1)$ 接收一个数据块。
3. 每步目标地址由 $(rank - step) \bmod size$ 计算，保证 $size - 1$ 步后所有数据到达所有进程。

Homework 6：思政思考——华为在高性能/并行计算领域的核心竞争力

以下为思政作业原文：

```
1 # 思政思考：华为在高性能/并行计算领域的核心竞争力
2
3 ## 与并行计算课程的联系
4
5 华为在 HPC/并行计算领域最大的竞争力在于
6 "芯片—框架—编译器"全栈自研的并行计算生态，与课程内容直接对应：
7
8 | 课程概念 | 华为对应能力 |
9 |-----|-----|
10 | MPI 集体通信（Gather、Allgather、Reduce 等） | MindSpore 框架内置的分布式并行策略，自动在集群中完成梯度 Allreduce、参数 Gather 等操作 |
11 | 进程间通信、拓扑结构 | HCCS（华为高速互联）实现昇腾芯片间低延迟通信，类似 MPI 中 communicator 的物理层实现 |
12 | 并行效率、负载均衡 | 昇腾 Ascend 的达芬奇架构（DaVinci Core）专为矩阵运算并行设计，3D Cube 单元单次可完成 16x16x16 的矩阵乘加 |
13 | 编译器优化 | 毕昇编译器（Bisheng）针对鲲鹏/昇腾自动向量化、循环并行化 |
14
15 ## 华为独特壁垒
16
17 相比 MPI 只定义"通信标准"的开放模式，华为的独特竞争力在于软硬协同：
18
19 - 鲲鹏处理器提供 ARM 架构的通用算力
20 - 昇腾 AI 处理器提供专用并行加速（达芬奇架构的 3D Cube）
21 - MindSpore 自动并行策略编排，开发者无需手动写 MPI 通信代码
22 - 毕昇编译器针对华为硬件深度优化
23
24 三者全栈打通，从芯片到并行框架全面自主可控。
```

8 附录：课程项目源码

本附录收录课程项目实验的三种并行进化算法完整 C++ 源码。三种算法均针对 TSPLIB 标准测试实例 pcb442（442 城市）设计，使用相同的进化操作框架（锦标赛选择、顺序交叉、交换变异、精英保留），仅在并行架构上有所区别。

Algorithm 1：分布式进化算法（dEA）

文件：TSP_dEA.cpp

架构：主从式（Master-Worker），4 进程

核心思想：Master（rank 0）维护全局 200 个个体的种群，每代广播全部个体至所有 Worker 进程进行适应度评估，评估结果通过 MPI_Gatherv 收集回 Master 进行选择 and 遗传操作。

```

1 // Algorithm 1: 分布式进化算法 (Distributed Evolutionary Algorithm)
2 // Master-worker model: rank 0 broadcasts the full population each generation,
3 // each worker mutates a chunk, results are gathered back, master selects.
4 // Supports multiple runs with result logging for statistical analysis.
5 // Compile: cl /EHsc /I"%MSMPI_INC%" TSP_dEA.cpp /link /LIBPATH:"%MSMPI_LIB64%" msmapi.lib
6 // Run: mpiexec -n 4 TSP_dEA.exe
7
8 #include <mpi.h>
9 #include <math.h>
10 #include <time.h>
11 #include <stdlib.h>
12 #include <stdio.h>
13 #include <string.h>
14
15 #define N_COLONY 100
16 #define CITY 442
17
18 int xColony = 100;
19 int xCity = CITY;
20 double probab1 = 0.02;
21 long maxGen = 200000;
22
23 int colony[N_COLONY * 2][CITY];
24 double city_dis[CITY][CITY];
25 double dis_p[N_COLONY * 2];
26 int temp[CITY];
27 double sumbest;
28 long GenNum;
29 double timeStart;
30
31 int position(int *tmp, int C);
32 void invert(int pos_start, int pos_end);
33 void printResult(long gen, double best, double elapsed, FILE *fp);
34 void select1();
35
36 int main(int argc, char **argv)
37 {
38     int rank, size;
39     MPI_Init(&argc, &argv);
40     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
41     MPI_Comm_size(MPI_COMM_WORLD, &size);
42

```

```

43 // Determine number of runs
44 int numRuns = 10;
45 if (argc > 1) numRuns = atoi(argv[1]);
46 if (argc > 2) maxGen = atol(argv[2]);
47
48 timeStart = MPI_Wtime();
49 srand((unsigned)time(NULL) + rank);
50
51 // ----- rank 0 reads city data -----
52 double cityXY[CITY][2];
53 if (rank == 0)
54 {
55     FILE *fp;
56     if ((fp = fopen("pcb442.tsp", "r")) == NULL)
57     {
58         printf("Rank 0: cannot open pcb442.tsp\n");
59         MPI_Abort(MPI_COMM_WORLD, 1);
60     }
61     fscanf(fp, "%d", &xCity);
62     for (int i = 0; i < xCity; i++)
63         fscanf(fp, "%d%lf%lf", &cityXY[i][0], &cityXY[i][1]);
64     fclose(fp);
65 }
66
67 MPI_Bcast(&xCity, 1, MPI_INT, 0, MPI_COMM_WORLD);
68 MPI_Bcast(cityXY, CITY * 2, MPI_DOUBLE, 0, MPI_COMM_WORLD);
69
70 // ----- all compute distance matrix -----
71 for (int i = 0; i < xCity; i++)
72     for (int j = 0; j < xCity; j++)
73     {
74         if (j > i)
75         {
76             double d = (cityXY[i][0] - cityXY[j][0]) * (cityXY[i][0] - cityXY[j][0])
77                 + (cityXY[i][1] - cityXY[j][1]) * (cityXY[i][1] - cityXY[j][1]);
78             city_dis[i][j] = (int)(sqrt(d) + 0.5);
79         }
80         else if (j == i) city_dis[i][j] = 0;
81         else city_dis[i][j] = city_dis[j][i];
82     }
83
84 // Division of work among processes
85 int chunk = xColony / size;
86 int start = rank * chunk;
87 int end = (rank == size - 1) ? xColony : start + chunk;
88 int local_count = end - start;
89
90 // Prepare Gatherv arrays on root
91 int *recv_counts = NULL, *displs = NULL;
92 int *recv_counts_d = NULL, *displs_d = NULL;
93 if (rank == 0)
94 {
95     recv_counts = (int *)malloc(size * sizeof(int));
96     displs = (int *)malloc(size * sizeof(int));
97     recv_counts_d = (int *)malloc(size * sizeof(int));
98     displs_d = (int *)malloc(size * sizeof(int));
99     for (int i = 0; i < size; i++)
100     {
101         int c = (i == size - 1) ? (xColony - i * chunk) : chunk;

```

```

102         recv_counts[i] = c * CITY;
103         displs[i] = i * chunk * CITY;
104         recv_counts_d[i] = c;
105         displs_d[i] = i * chunk;
106     }
107 }
108
109 // Results file (only rank 0)
110 FILE *resFile = NULL;
111 if (rank == 0)
112 {
113     resFile = fopen("results_dEA.txt", "w");
114     if (resFile)
115     {
116         fprintf(resFile, "# Algorithm 1: dEA (Distributed Evolutionary Algorithm)\n");
117         fprintf(resFile, "# pcb442.tsp, population=%d, maxGen=%ld, processes=%d\n",
118                 xColony, maxGen, size);
119         fprintf(resFile, "# Run\tBest\tTime(s)\n");
120     }
121 }
122
123 // ----- multi-run loop -----
124 for (int run = 0; run < numRuns; run++)
125 {
126     if (rank == 0)
127     {
128         // Re-seed each run
129         srand((unsigned)time(NULL) + rank + run * 137);
130     }
131     MPI_Bcast(&run, 1, MPI_INT, 0, MPI_COMM_WORLD);
132
133     // ---- rank 0 initialises colony ----
134     if (rank == 0)
135     {
136         int array[CITY];
137         for (int i = 0; i < xCity; i++) array[i] = i;
138
139         for (int i = 0; i < xColony; i++)
140         {
141             int mod = xCity;
142             for (int j = 0; j < xCity; j++)
143             {
144                 int sign = rand() % mod;
145                 colony[i][j] = array[sign];
146                 int t = array[mod - 1];
147                 array[mod - 1] = array[sign];
148                 array[sign] = t;
149                 mod--;
150                 if (mod == 1) { j++; colony[i][j] = array[0]; break; }
151             }
152         }
153
154         for (int i = 0; i < xColony; i++)
155         {
156             dis_p[i] = 0;
157             for (int j = 0; j < xCity - 1; j++)
158                 dis_p[i] += city_dis[colony[i][j]][colony[i][j + 1]];
159             dis_p[i] += city_dis[colony[i][0]][colony[i][xCity - 1]];
160         }
161     }
162 }

```



```

160
161     sumbest = dis_p[0];
162     for (int i = 1; i < xColony; i++)
163         if (dis_p[i] < sumbest) sumbest = dis_p[i];
164
165     if (run == 0) printf("init success! initial best = %f\n", sumbest);
166 }
167
168 double runStart = MPI_Wtime();
169
170 // ---- generation loop ----
171 for (GenNum = 0; GenNum < maxGen; GenNum++)
172 {
173     MPI_Bcast(colony, N_COLONY * CITY, MPI_INT, 0, MPI_COMM_WORLD);
174     MPI_Bcast(dis_p, N_COLONY, MPI_DOUBLE, 0, MPI_COMM_WORLD);
175
176     // Each process mutates its chunk
177     for (int i = start; i < end; i++)
178     {
179         for (int j = 0; j < xCity; j++)
180             temp[j] = colony[i][j];
181
182         double disChange = 0;
183         int pos_flag = 0;
184         int pos_C = rand() % xCity;
185
186         for (;;)
187         {
188             int pos_C1, C1, j, k1, k2, l1, l2;
189
190             if ((rand() / 32768.0) < probab1)
191             {
192                 do pos_C1 = rand() % xCity;
193                 while (pos_C1 == pos_C);
194                 C1 = colony[i][pos_C1];
195             }
196             else
197             {
198                 do j = rand() % xColony;
199                 while (j == i);
200                 int k = position(colony[j], temp[pos_C]);
201                 C1 = colony[j][(k + 1) % xCity];
202                 pos_C1 = position(temp, C1);
203             }
204
205             if ((pos_C + 1) % xCity == pos_C1 || (pos_C - 1 + xCity) % xCity == pos_C1)
206                 break;
207
208             k1 = temp[pos_C];
209             k2 = temp[(pos_C + 1) % xCity];
210             l1 = temp[pos_C1];
211             l2 = temp[(pos_C1 + 1) % xCity];
212             disChange += city_dis[k1][l1] + city_dis[k2][l2]
213                         - city_dis[k1][k2] - city_dis[l1][l2];
214
215             invert(pos_C, pos_C1);
216             pos_flag++;
217             if (pos_flag > xCity - 1)

```

```

218         break;
219         pos_C++;
220         if (pos_C >= xCity) pos_C = 0;
221     }
222
223     dis_p[N_COLONY + i] = dis_p[i] + disChange;
224     for (int j = 0; j < xCity; j++)
225         colony[N_COLONY + i][j] = temp[j];
226 }
227
228 // Gather children to root
229 if (rank == 0)
230 {
231     MPI_Gatherv(MPI_IN_PLACE, 0, MPI_DATATYPE_NULL,
232                 &colony[N_COLONY][0], recv_counts, displs, MPI_INT,
233                 0, MPI_COMM_WORLD);
234     MPI_Gatherv(MPI_IN_PLACE, 0, MPI_DATATYPE_NULL,
235                 &dis_p[N_COLONY], recv_counts_d, displs_d, MPI_DOUBLE,
236                 0, MPI_COMM_WORLD);
237 }
238 else
239 {
240     MPI_Gatherv(&colony[N_COLONY + start][0], local_count * CITY, MPI_INT,
241                 NULL, NULL, NULL, MPI_INT, 0, MPI_COMM_WORLD);
242     MPI_Gatherv(&dis_p[N_COLONY + start], local_count, MPI_DOUBLE,
243                 NULL, NULL, NULL, MPI_DOUBLE, 0, MPI_COMM_WORLD);
244 }
245
246 // Root: select
247 if (rank == 0)
248 {
249     select1();
250     sumbest = dis_p[0];
251     for (int j = 1; j < xColony; j++)
252         if (dis_p[j] < sumbest) sumbest = dis_p[j];
253 }
254 }
255
256 // ---- end of run ----
257 if (rank == 0)
258 {
259     double runTime = MPI_Wtime() - runStart;
260     printf("Run %d: best=%f, time=%.2f s\n", run + 1, sumbest, runTime);
261     if (resFile)
262         fprintf(resFile, "%d\t%.0f\t%.2f\n", run + 1, sumbest, runTime);
263 }
264 }
265
266 // Final output
267 if (rank == 0)
268 {
269     double totalTime = MPI_Wtime() - timeStart;
270     printf("Total time: %.2f s (%d runs)\n", totalTime, numRuns);
271     if (resFile) fclose(resFile);
272 }
273
274 free(recv_counts); free(displs);
275 free(recv_counts_d); free(displs_d);
276 MPI_Finalize();

```

```

277     return 0;
278 }
279
280 void select1()
281 {
282     for (int j = 0; j < N_COLONY; j++)
283         if (dis_p[N_COLONY + j] < dis_p[j])
284             {
285                 dis_p[j] = dis_p[N_COLONY + j];
286                 for (int k = 0; k < CITY; k++)
287                     colony[j][k] = colony[N_COLONY + j][k];
288             }
289 }
290
291 void invert(int pos_start, int pos_end)
292 {
293     int j, k, t;
294     if (pos_start < pos_end)
295     {
296         j = pos_start + 1;
297         k = pos_end;
298         for (; j <= k; j++, k--)
299             { t = temp[j]; temp[j] = temp[k]; temp[k] = t; }
300     }
301     else
302     {
303         if (xCity - 1 - pos_start <= pos_end + 1)
304         {
305             j = pos_end; k = pos_start + 1;
306             for (; k < xCity; j--, k++)
307                 { t = temp[j]; temp[j] = temp[k]; temp[k] = t; }
308             k = 0;
309             for (; k <= j; k++, j--)
310                 { t = temp[j]; temp[j] = temp[k]; temp[k] = t; }
311         }
312         else
313         {
314             j = pos_end; k = pos_start + 1;
315             for (; j >= 0; j--, k++)
316                 { t = temp[j]; temp[j] = temp[k]; temp[k] = t; }
317             j = xCity - 1;
318             for (; k <= j; k++, j--)
319                 { t = temp[j]; temp[j] = temp[k]; temp[k] = t; }
320         }
321     }
322 }
323
324 int position(int *tmp, int C)
325 {
326     for (int j = 0; j < xCity; j++)
327         if (tmp[j] == C) return j;
328     return -1;
329 }

```

Algorithm 2: 分层分布式进化算法 (HdEA)

文件: TSP_HdEA.cpp

架构: 岛屿模型 + 两层分层, 4 进程

核心思想: 每个进程独立维护 50 个个体的子种群进行进化, 每 20 代各进程将本岛最优的 3 个精英发送至 Master 进行二阶精英进化, 进化后的全局最优通过广播分发回各岛屿。

```

1 // Algorithm 2: 分层分布式进化算法 (Hierarchical Distributed EA)
2 //
3 // Two-level hierarchy:
4 //   Level 1 (islands): each rank evolves its own independent subpopulation
5 //   Level 2 (master): rank 0 collects best individuals from all islands,
6 //                       runs additional evolution on the elite pool, and
7 //                       broadcasts the global best back to all islands.
8 //
9 // Compile: cl /EHsc /I"%MSMPI_INC%" TSP_HdEA.cpp /link /LIBPATH:"%MSMPI_LIB64%" msmapi.lib
10 // Run: mpiexec -n 4 TSP_HdEA.exe [runs] [maxGen]
11
12 #include <mpi.h>
13 #include <math.h>
14 #include <time.h>
15 #include <stdlib.h>
16 #include <stdio.h>
17 #include <string.h>
18
19 #define CITY    442
20 #define POP     50
21 #define ELITE   50
22 #define MIGRATE 50
23 #define MASTER_GENS 20
24
25 static int xCity = CITY;
26 static double probabl = 0.02;
27 static long maxGen = 200000;
28 static double city_dis[CITY][CITY];
29 static int temp[CITY];
30
31 static int position(int *t, int c);
32 static void invert(int a, int b);
33 static double dist(int *t);
34 static void mutate(int *parent, int *child, double pDis, double *cDis,
35                  int (*pop)[CITY], double *dps, int ps, int idx);
36 static void evolve_pop(int (*pop)[CITY], double *dps, int ps);
37 static int best_idx(double *dps, int ps);
38 static int worst_idx(double *dps, int ps);
39 static void init_rand_tour(int *tour);
40
41 int main(int argc, char **argv)
42 {
43     int rk, np;
44     MPI_Init(&argc, &argv);
45     MPI_Comm_rank(MPI_COMM_WORLD, &rk);
46     MPI_Comm_size(MPI_COMM_WORLD, &np);
47
48     int runs = 10;
49     if (argc > 1) runs = atoi(argv[1]);
50     if (argc > 2) maxGen = atol(argv[2]);
51     srand((unsigned)time(NULL) + rk);

```

```

52
53  /* read city data */
54  double xy[CITY][2];
55  if (rk == 0) {
56      FILE *f = fopen("pcb442.tsp", "r");
57      if (!f) { printf("cannot open pcb442.tsp\n"); MPI_Abort(MPI_COMM_WORLD, 1); }
58      fscanf(f, "%d", &xCity);
59      for (int i = 0; i < xCity; i++) fscanf(f, "%d%lf%lf", &xy[i][0], &xy[i][1]);
60      fclose(f);
61  }
62  MPI_Bcast(&xCity, 1, MPI_INT, 0, MPI_COMM_WORLD);
63  MPI_Bcast(xy, CITY*2, MPI_DOUBLE, 0, MPI_COMM_WORLD);
64
65  for (int i = 0; i < xCity; i++)
66      for (int j = 0; j < xCity; j++) {
67          if (j > i) {
68              double d = (xy[i][0]-xy[j][0])*(xy[i][0]-xy[j][0])
69                      + (xy[i][1]-xy[j][1])*(xy[i][1]-xy[j][1]);
70              city_dis[i][j] = (int)(sqrt(d)+0.5);
71          } else if (j == i) city_dis[i][j] = 0;
72          else city_dis[i][j] = city_dis[j][i];
73      }
74
75  int colony[POP][CITY];
76  double dis[POP];
77  int elite[ELITE][CITY];
78  double eliteDis[ELITE];
79
80  FILE *log = NULL;
81  if (rk == 0) {
82      log = fopen("results_HdEA.txt", "w");
83      if (log) fprintf(log, "# HdEA run\\tbest\\ttime\\n");
84      printf("HdEA: %d procs, %d runs, %ld gens, POP=%d\\n", np, runs, maxGen, POP);
85  }
86
87  double t0 = MPI_Wtime();
88  MPI_Datatype MPI_TOUR;
89  MPI_Type_contiguous(CITY, MPI_INT, &MPI_TOUR);
90  MPI_Type_commit(&MPI_TOUR);
91
92  for (int run = 0; run < runs; run++) {
93      srand((unsigned)time(NULL) + rk + run*137);
94      double run0 = MPI_Wtime();
95
96      /* init island with valid random tours */
97      for (int i = 0; i < POP; i++) {
98          init_rand_tour(colony[i]);
99          dis[i] = dist(colony[i]);
100      }
101      /* init elite with valid random tours too */
102      for (int i = 0; i < ELITE; i++) {
103          init_rand_tour(elite[i]);
104          eliteDis[i] = dist(elite[i]);
105      }
106
107      for (long gen = 0; gen < maxGen; gen++) {
108          evolve_pop(colony, dis, POP);
109
110          if (gen > 0 && gen % MIGRATE == 0) {

```

```

111         int bi = best_idx(dis, POP);
112
113         /* workers send best to master */
114         if (rk != 0) {
115             MPI_Send(colony[bi], 1, MPI TOUR, 0, 10, MPI_COMM_WORLD);
116             MPI_Send(&dis[bi], 1, MPI_DOUBLE, 0, 11, MPI_COMM_WORLD);
117         } else {
118             /* master: receive best from each worker */
119             int recvTour[CITY];
120             double recvD;
121             for (int i = 1; i < np; i++) {
122                 MPI_Recv(recvTour, 1, MPI TOUR, i, 10, MPI_COMM_WORLD,
123                     MPI_STATUS_IGNORE);
124                 MPI_Recv(&recvD, 1, MPI_DOUBLE, i, 11, MPI_COMM_WORLD,
125                     MPI_STATUS_IGNORE);
126                 int wi = worst_idx(eliteDis, ELITE);
127                 if (recvD < eliteDis[wi]) {
128                     eliteDis[wi] = recvD;
129                     memcpy(elite[wi], recvTour, CITY*sizeof(int));
130                 }
131             }
132             /* also inject master's own best into elite */
133             if (dis[bi] < eliteDis[worst_idx(eliteDis, ELITE)]) {
134                 int wi = worst_idx(eliteDis, ELITE);
135                 eliteDis[wi] = dis[bi];
136                 memcpy(elite[wi], colony[bi], CITY*sizeof(int));
137             }
138             /* evolve elite on master */
139             for (int mg = 0; mg < MASTER_GENS; mg++)
140                 evolve_pop(elite, eliteDis, ELITE);
141
142             /* all ranks: wait for global best broadcast */
143             int gTour[CITY];
144             double gBest;
145             if (rk == 0) {
146                 int gbi = best_idx(eliteDis, ELITE);
147                 memcpy(gTour, elite[gbi], CITY*sizeof(int));
148                 gBest = eliteDis[gbi];
149             }
150             MPI_Bcast(gTour, 1, MPI TOUR, 0, MPI_COMM_WORLD);
151             MPI_Bcast(&gBest, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
152
153             /* inject global best into local island */
154             int wi = worst_idx(dis, POP);
155             if (gBest < dis[wi]) {
156                 dis[wi] = gBest;
157                 memcpy(colony[wi], gTour, CITY*sizeof(int));
158             }
159         }
160
161         /* collect global best */
162         double lb = dis[best_idx(dis, POP)];
163         double gb;
164         MPI_Reduce(&lb, &gb, 1, MPI_DOUBLE, MPI_MIN, 0, MPI_COMM_WORLD);
165         if (rk == 0) {
166             double eb = eliteDis[best_idx(eliteDis, ELITE)];
167             if (eb < gb) gb = eb;

```

```

168         double t = MPI_Wtime() - run0;
169         printf("Run %d: best=%.0f time=%.2f\n", run+1, gb, t);
170         if (log) fprintf(log, "%d\t%.0f\t%.2f\n", run+1, gb, t);
171     }
172 }
173
174 if (rk == 0) {
175     printf("Total time: %.2f s\n", MPI_Wtime() - t0);
176     if (log) fclose(log);
177 }
178 MPI_Type_free(&MPI_TOUR);
179 MPI_Finalize();
180 return 0;
181 }
182
183 /* ---- helpers ---- */
184
185 static void init_rand_tour(int *tour) {
186     int arr[CITY];
187     for (int j = 0; j < xCity; j++) arr[j] = j;
188     int m = xCity;
189     for (int j = 0; j < xCity; j++) {
190         int s = rand() % m;
191         tour[j] = arr[s];
192         arr[s] = arr[m-1];
193         if (--m == 1) { tour[++j] = arr[0]; break; }
194     }
195 }
196
197 static double dist(int *t) {
198     double d = 0;
199     for (int j = 0; j < xCity-1; j++) d += city_dis[t[j]][t[j+1]];
200     return d + city_dis[t[0]][t[xCity-1]];
201 }
202
203 static int position(int *t, int c) {
204     for (int j = 0; j < xCity; j++) if (t[j] == c) return j;
205     return -1;
206 }
207
208 static void invert(int a, int b) {
209     if (a < b) {
210         for (int i = a+1, j = b; i <= j; i++, j--)
211             { int t = temp[i]; temp[i] = temp[j]; temp[j] = t; }
212     } else if (xCity-1-a <= b+1) {
213         int i = b, j = a+1;
214         for (; j < xCity; i--, j++) { int t = temp[i]; temp[i] = temp[j]; temp[j] = t; }
215         for (j = 0; j <= i; j++, i--) { int t = temp[i]; temp[i] = temp[j]; temp[j] = t; }
216     } else {
217         int i = b, j = a+1;
218         for (; i >= 0; i--, j++) { int t = temp[i]; temp[i] = temp[j]; temp[j] = t; }
219         for (i = xCity-1; j <= i; j++, i--) { int t = temp[i]; temp[i] = temp[j]; temp[j] = t; }
220     }
221 }
222
223 static void mutate(int *parent, int *child, double pDis, double *cDis,
224                  int (*pop)[CITY], double *dps, int ps, int idx)
225 {

```

```

226     memcpy(temp, parent, CITY*sizeof(int));
227     double ch = 0;
228     int n = 0, pc = rand() % xCity;
229     for (;;) {
230         int pc1, c1, j;
231         if ((rand()/32768.0) < probabl) {
232             do pc1 = rand() % xCity; while (pc1 == pc);
233             c1 = pop[idx][pc1];
234         } else {
235             do j = rand() % ps; while (j == idx);
236             int k = position(pop[j], temp[pc]);
237             if (k < 0) break;
238             c1 = pop[j][(k+1)%xCity];
239             pc1 = position(temp, c1);
240             if (pc1 < 0) break;
241         }
242         if ((pc+1)%xCity == pc1 || (pc-1+xCity)%xCity == pc1) break;
243         int k1 = temp[pc], k2 = temp[(pc+1)%xCity];
244         int l1 = temp[pc1], l2 = temp[(pc1+1)%xCity];
245         ch += city_dis[k1][l1] + city_dis[k2][l2] - city_dis[k1][k2] - city_dis[l1][l2];
246         invert(pc, pc1);
247         if (++n > xCity-1) break;
248         if (++pc >= xCity) pc = 0;
249     }
250     *cDis = pDis + ch;
251     memcpy(child, temp, CITY*sizeof(int));
252 }
253
254 static void evolve_pop(int (*pop)[CITY], double *dps, int ps) {
255     int off[POP][CITY];
256     double offDis[POP];
257     for (int i = 0; i < ps; i++)
258         mutate(pop[i], off[i], dps[i], &offDis[i], pop, dps, ps, i);
259     for (int i = 0; i < ps; i++)
260         if (offDis[i] < dps[i]) {
261             dps[i] = offDis[i];
262             memcpy(pop[i], off[i], CITY*sizeof(int));
263         }
264 }
265
266 static int best_idx(double *dps, int ps) {
267     int bi = 0;
268     for (int i = 1; i < ps; i++) if (dps[i] < dps[bi]) bi = i;
269     return bi;
270 }
271
272 static int worst_idx(double *dps, int ps) {
273     int wi = 0;
274     for (int i = 1; i < ps; i++) if (dps[i] > dps[wi]) wi = i;
275     return wi;
276 }

```


Algorithm 3: 带种群迁移的分层分布式进化算法 (HdEA-MP)

文件: TSP_HdEA_MP.cpp

架构: 岛屿模型 + 环形迁移, 4 进程

核心思想: 在 HdEA 的精英协调机制之上, 每 20 代额外执行一次环形迁移——每个进程将其最优的 3 个个体通过 MPI_Sendrecv 发送至右邻居进程, 并接收来自左邻居的移民, 用更优的移民替换本地最差个体, 以增加岛屿之间的基因交流。

```

1 // Algorithm 3: 移动种群分层分布式进化算法 (HdEA with Migration Population)
2 //
3 // Extends Algorithm 2 (HdEA) with peer-to-peer migration between islands
4 // in a ring topology. In addition to the hierarchical elite exchange:
5 //   - Every MIGRATION_INTERVAL, each rank sends its M = 3 best individuals
6 //     to the next rank in a ring (rank i -> rank (i+1)%np)
7 //   - Each rank receives M individuals from the previous rank
8 //   - Received migrants replace the worst individuals in the island
9 //
10 // The hierarchy ensures global convergence pressure; migration ensures
11 // diversity flow between islands, preventing premature convergence.
12 //
13 // Compile: cl /EHsc /I"%MSMPI_INC%" TSP_HdEA_MP.cpp /link /LIBPATH:"%MSMPI_LIB64%" msmapi.lib
14 // Run: mpiexec -n 4 TSP_HdEA_MP.exe
15
16 #include <mpi.h>
17 #include <math.h>
18 #include <time.h>
19 #include <stdlib.h>
20 #include <stdio.h>
21 #include <string.h>
22
23 #define CITY      442
24 #define POP       50      // individuals per island
25 #define ELITE     50      // elite population size on master
26 #define MIGRATE   50      // migration interval (generations)
27 #define MASTER_GENS 20    // master evolution gens per interval
28 #define MIG_M     3       // number of migrants per exchange
29
30 static int xCity = CITY;
31 static double probab1 = 0.02;
32 static long maxGen = 200000;
33 static double city_dis[CITY][CITY];
34 static int temp[CITY];
35
36 static int position(int *t, int c);
37 static void invert(int a, int b);
38 static double dist(int *t);
39 static void mutate(int *parent, int *child, double pDis, double *cDis,
40                  int (*colony)[CITY], double *dps, int ps, int idx);
41 static void evolve_island(int (*colony)[CITY], double *dps, int ps);
42 static int best_idx(double *dps, int ps);
43 static int worst_idx(double *dps, int ps);
44 static void sort_by_fitness(int (*colony)[CITY], double *dps, int ps);
45
46 int main(int argc, char **argv)
47 {
48     int rk, np;
49     MPI_Init(&argc, &argv);
50     MPI_Comm_rank(MPI_COMM_WORLD, &rk);
51     MPI_Comm_size(MPI_COMM_WORLD, &np);

```

```

52
53     int runs = 10;
54     if (argc > 1) runs = atoi(argv[1]);
55     if (argc > 2) maxGen = atol(argv[2]);
56     srand((unsigned)time(NULL) + rk);
57
58     /* ---- read city file on root ---- */
59     double xy[CITY][2];
60     if (rk == 0) {
61         FILE *f = fopen("pcb442.tsp", "r");
62         if (!f) { printf("cannot open pcb442.tsp\n"); MPI_Abort(MPI_COMM_WORLD, 1); }
63         fscanf(f, "%d", &xCity);
64         for (int i = 0; i < xCity; i++) fscanf(f, "%d%lf%lf", &xy[i][0], &xy[i][1]);
65         fclose(f);
66     }
67     MPI_Bcast(&xCity, 1, MPI_INT, 0, MPI_COMM_WORLD);
68     MPI_Bcast(xy, CITY*2, MPI_DOUBLE, 0, MPI_COMM_WORLD);
69
70     /* ---- distance matrix ---- */
71     for (int i = 0; i < xCity; i++)
72         for (int j = 0; j < xCity; j++) {
73             if (j > i) {
74                 double d = (xy[i][0]-xy[j][0])*(xy[i][0]-xy[j][0])
75                     + (xy[i][1]-xy[j][1])*(xy[i][1]-xy[j][1]);
76                 city_dis[i][j] = (int)(sqrt(d)+0.5);
77             } else if (j == i) city_dis[i][j] = 0;
78             else city_dis[i][j] = city_dis[j][i];
79         }
80
81     /* populations */
82     int colony[POP][CITY];
83     double dis[POP];
84     int elite[ELITE][CITY];
85     double eliteDis[ELITE];
86
87     /* results */
88     FILE *log = NULL;
89     if (rk == 0) {
90         log = fopen("results_HdEA_MP.txt", "w");
91         if (log) fprintf(log, "# HdEA-MP run\\tbest\\tttime\\n");
92     }
93
94     double t0 = MPI_Wtime();
95     MPI_Datatype MPI_TOUR;
96     MPI_Type_contiguous(CITY, MPI_INT, &MPI_TOUR);
97     MPI_Type_commit(&MPI_TOUR);
98
99     /* ---- multi-run ---- */
100    for (int run = 0; run < runs; run++) {
101        srand((unsigned)time(NULL) + rk + run*137);
102        double run0 = MPI_Wtime();
103
104        /* init island & elite with valid random tours */
105        for (int i = 0; i < POP; i++) {
106            int arr[CITY];
107            for (int j = 0; j < xCity; j++) arr[j] = j;
108            int m = xCity;
109            for (int j = 0; j < xCity; j++) {
110                int s = rand() % m;

```

```

111         colony[i][j] = arr[s];
112         arr[s] = arr[m-1];
113         if (--m == 1) { colony[i][++j] = arr[0]; break; }
114     }
115     dis[i] = dist(colony[i]);
116 }
117 for (int i = 0; i < ELITE; i++) {
118     int arr[CITY];
119     for (int j = 0; j < xCity; j++) arr[j] = j;
120     int m = xCity;
121     for (int j = 0; j < xCity; j++) {
122         int s = rand() % m;
123         elite[i][j] = arr[s];
124         arr[s] = arr[m-1];
125         if (--m == 1) { elite[i][++j] = arr[0]; break; }
126     }
127     eliteDis[i] = dist(elite[i]);
128 }
129
130 /* evolve */
131 for (long gen = 0; gen < maxGen; gen++) {
132     evolve_island(colony, dis, POP);
133
134     if (gen > 0 && gen % MIGRATE == 0) {
135         /* ===== Step 1: Hierarchical elite exchange ===== */
136         int bi = best_idx(dis, POP);
137
138         /* Point-to-point: send best to master */
139         if (rk != 0) {
140             MPI_Send(colony[bi], 1, MPI TOUR, 0, 10, MPI_COMM_WORLD);
141             MPI_Send(&dis[bi], 1, MPI_DOUBLE, 0, 11, MPI_COMM_WORLD);
142         } else {
143             int recvTour[CITY];
144             double recvDis;
145             for (int i = 1; i < np; i++) {
146                 MPI_Recv(recvTour, 1, MPI TOUR, i, 10, MPI_COMM_WORLD,
147                     MPI_STATUS_IGNORE);
148                 MPI_Recv(&recvDis, 1, MPI_DOUBLE, i, 11, MPI_COMM_WORLD,
149                     MPI_STATUS_IGNORE);
150                 int wi = worst_idx(eliteDis, ELITE);
151                 if (recvDis < eliteDis[wi]) {
152                     eliteDis[wi] = recvDis;
153                     memcpy(elite[wi], recvTour, CITY*sizeof(int));
154                 }
155             }
156             int mbi = best_idx(dis, POP);
157             int wi = worst_idx(eliteDis, ELITE);
158             if (dis[mbi] < eliteDis[wi]) {
159                 eliteDis[wi] = dis[mbi];
160                 memcpy(elite[wi], colony[mbi], CITY*sizeof(int));
161             }
162
163             for (int mg = 0; mg < MASTER_GENS; mg++)
164                 evolve_island(elite, eliteDis, ELITE);
165         }
166
167         /* Collective: master broadcasts global best elite */
168         int gTour[CITY];
169         double gBest;

```

```

168     if (rk == 0) {
169         int gbi = best_idx(eliteDis, ELITE);
170         memcpy(gTour, elite[gbi], CITY*sizeof(int));
171         gBest = eliteDis[gbi];
172     }
173     MPI_Bcast(gTour, 1, MPI_TOUR, 0, MPI_COMM_WORLD);
174     MPI_Bcast(&gBest, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
175
176     int wi = worst_idx(dis, POP);
177     if (gBest < dis[wi]) {
178         dis[wi] = gBest;
179         memcpy(colony[wi], gTour, CITY*sizeof(int));
180     }
181
182     /* ===== Step 2: Peer-to-peer ring migration (NEW in Alg 3) ===== */
183     sort_by_fitness(colony, dis, POP);
184
185     int sendTo = (rk + 1) % np;
186     int recvFrom = (rk - 1 + np) % np;
187
188     /* Pack all migrants into flat buffer: [MIG_M tours][MIG_M doubles] */
189     int packSize = MIG_M * (CITY + 1);
190     int *sendPacked = (int *)malloc(packSize * sizeof(int));
191     int *recvPacked = (int *)malloc(packSize * sizeof(int));
192     for (int m = 0; m < MIG_M && m < POP; m++) {
193         memcpy(&sendPacked[m * CITY], colony[m], CITY * sizeof(int));
194         sendPacked[MIG_M * CITY + m] = (int)dis[m];
195     }
196
197     /* Ring migration via MPI_Sendrecv (avoids buffering issues) */
198     MPI_Sendrecv(sendPacked, packSize, MPI_INT, sendTo, 100,
199                 recvPacked, packSize, MPI_INT, recvFrom, 100,
200                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
201
202     for (int m = 0; m < MIG_M; m++) {
203         int wii = worst_idx(dis, POP);
204         double recvD = (double)recvPacked[MIG_M * CITY + m];
205         if (recvD < dis[wii]) {
206             dis[wii] = recvD;
207             memcpy(colony[wii], &recvPacked[m * CITY], CITY * sizeof(int));
208         }
209     }
210
211     free(sendPacked);
212     free(recvPacked);
213 }
214
215 if (gen % 20000 == 0 && rk == 0) {
216     double bi = dis[best_idx(dis, POP)];
217     double gi = eliteDis[best_idx(eliteDis, ELITE)];
218     double b = bi < gi ? bi : gi;
219     printf("gen %ld best=%.0f\n", gen, b);
220 }
221 }
222
223 /* collect global best */
224 double lb = dis[best_idx(dis, POP)];
225 double gb;
226 MPI_Reduce(&lb, &gb, 1, MPI_DOUBLE, MPI_MIN, 0, MPI_COMM_WORLD);

```

```

227     if (rk == 0) {
228         double eb = eliteDis[best_idx(eliteDis, ELITE)];
229         if (eb < gb) gb = eb;
230         double t = MPI_Wtime() - run0;
231         printf("Run %d: best=%.0f time=%.2f\n", run+1, gb, t);
232         if (log) fprintf(log, "%d\t%.0f\t%.2f\n", run+1, gb, t);
233     }
234 }
235
236 if (rk == 0) {
237     printf("Total time: %.2f s\n", MPI_Wtime() - t0);
238     if (log) fclose(log);
239 }
240 MPI_Type_free(&MPI_TOUR);
241 MPI_Finalize();
242 return 0;
243 }
244
245 /* ---- helpers ---- */
246 static double dist(int *t) {
247     double d = 0;
248     for (int j = 0; j < xCity-1; j++) d += city_dis[t[j]][t[j+1]];
249     return d + city_dis[t[0]][t[xCity-1]];
250 }
251
252 static int position(int *t, int c) {
253     for (int j = 0; j < xCity; j++) if (t[j] == c) return j;
254     return -1;
255 }
256
257 static void invert(int a, int b) {
258     if (a < b) {
259         for (int i = a+1, j = b; i <= j; i++, j--)
260             { int t = temp[i]; temp[i] = temp[j]; temp[j] = t; }
261     } else if (xCity-1-a <= b+1) {
262         int i = b, j = a+1;
263         for (; j < xCity; i--, j++) { int t = temp[i]; temp[i] = temp[j]; temp[j] = t; }
264         for (j = 0; j <= i; j++, i--) { int t = temp[i]; temp[i] = temp[j]; temp[j] = t; }
265     } else {
266         int i = b, j = a+1;
267         for (; i >= 0; i--, j++) { int t = temp[i]; temp[i] = temp[j]; temp[j] = t; }
268         for (i = xCity-1; j <= i; j++, i--) { int t = temp[i]; temp[i] = temp[j]; temp[j] = t; }
269     }
270 }
271
272 static void mutate(int *parent, int *child, double pDis, double *cDis,
273                  int (*colony)[CITY], double *dps, int ps, int idx)
274 {
275     memcpy(temp, parent, CITY*sizeof(int));
276     double ch = 0;
277     int n = 0, pc = rand() % xCity;
278     for (;;) {
279         int pc1, c1, j, k1, k2, l1, l2;
280         if ((rand()/32768.0) < probab1) {
281             do pc1 = rand() % xCity; while (pc1 == pc);
282             c1 = colony[idx][pc1];
283         } else {
284             do j = rand() % ps; while (j == idx);

```

```

285         int k = position(colony[j], temp[pc]);
286         if (k < 0) break;
287         c1 = colony[j][(k+1)%xCity];
288         pc1 = position(temp, c1);
289         if (pc1 < 0) break;
290     }
291     if ((pc+1)%xCity == pc1 || (pc-1+xCity)%xCity == pc1) break;
292     k1 = temp[pc]; k2 = temp[(pc+1)%xCity];
293     l1 = temp[pc1]; l2 = temp[(pc1+1)%xCity];
294     ch += city_dis[k1][l1] + city_dis[k2][l2] - city_dis[k1][k2] - city_dis[l1][l2];
295     invert(pc, pc1);
296     if (++n > xCity-1) break;
297     if (++pc >= xCity) pc = 0;
298 }
299 *cDis = pDis + ch;
300 memcpy(child, temp, CITY*sizeof(int));
301 }
302
303 static void evolve_island(int (*colony)[CITY], double *dps, int ps) {
304     int off[POP][CITY];
305     double offDis[POP];
306     for (int i = 0; i < ps; i++)
307         mutate(colony[i], off[i], dps[i], &offDis[i], colony, dps, ps, i);
308     for (int i = 0; i < ps; i++)
309         if (offDis[i] < dps[i]) {
310             dps[i] = offDis[i];
311             memcpy(colony[i], off[i], CITY*sizeof(int));
312         }
313 }
314
315 static int best_idx(double *dps, int ps) {
316     int bi = 0;
317     for (int i = 1; i < ps; i++) if (dps[i] < dps[bi]) bi = i;
318     return bi;
319 }
320
321 static int worst_idx(double *dps, int ps) {
322     int wi = 0;
323     for (int i = 1; i < ps; i++) if (dps[i] > dps[wi]) wi = i;
324     return wi;
325 }
326
327 static void sort_by_fitness(int (*colony)[CITY], double *dps, int ps) {
328     /* Simple insertion sort: best (lowest distance) first */
329     for (int i = 1; i < ps; i++) {
330         double key = dps[i];
331         int keyTour[CITY];
332         memcpy(keyTour, colony[i], CITY*sizeof(int));
333         int j = i - 1;
334         while (j >= 0 && dps[j] > key) {
335             dps[j+1] = dps[j];
336             memcpy(colony[j+1], colony[j], CITY*sizeof(int));
337             j--;
338         }
339         dps[j+1] = key;
340         memcpy(colony[j+1], keyTour, CITY*sizeof(int));
341     }
342 }

```